

MAC Array

Design Specification

LONGHORN SILICON LLM INFERENCE ACCELERATOR
Block 2 of 4 — Attention Compute Unit (ACU), MAC engine

Version: `mac-array-v0.1` (first public draft)
Date: 2026-05-14
Status: Reference model complete; RTL pending
Author: Chaithu Talasila, The University of Texas at Austin
Reference: `LonghornSilicon/adaptive-precision-attention`

Scope. This document describes the externally-visible behavior of the MAC array block of the LonghornSilicon ACU. Decisions marked **frozen** will not be revisited in v0.2; decisions marked **open** may change as the RTL lands and the compiler team's requirements clarify. The accompanying bit-accurate C/C++/Python reference implementations are the executable form of this specification.

Contents

1	Overview	3
2	Operations Exposed	3
3	Data Type Semantics	3
3.1	INT8 Path (Frozen)	3
3.2	FP16 Path (Open — v0.1 Simplification)	3
3.3	Why Both Paths (Frozen)	4
4	Shape Support (Frozen)	4
5	Concurrency and Streaming (Open)	4
6	Cost Model (v0.1 Placeholders)	4
7	What Is Not in v0.1	5
8	Compiler Binding	5
9	Open Questions	6
10	Verification Status	6
11	Change Log	6

1. Overview

The MAC array is the compute engine of the ACU. After the precision controller (see `pc-isa-0.1`) decides INT8 or FP16 for a tile, the MAC array executes the chosen matmul. The compiler emits `matmul_int8` or `matmul_fp16` per tile based on the gate’s decision.

Functional summary. For row-major inputs $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, the block computes $C = A \cdot B \in \mathbb{R}^{M \times N}$ at either INT8 or FP16 precision. Output precision matches input precision in the FP16 path; output precision is widened to int32 in the INT8 path (the compiler is responsible for any downcast or requantization).

2. Operations Exposed

The block exposes four logical operations to the compiler:

Table 1: MAC array operations.

Operation	Inputs	Output	Use
<code>matmul_int8</code>	int8 $[M \times K]$, int8 $[K \times N]$	int32 $[M \times N]$	INT8 attention / MLP path
<code>matmul_fp16</code>	fp16 $[M \times K]$, fp16 $[K \times N]$	fp16 $[M \times N]$	FP16 attention / MLP path
<code>query</code>	—	<code>MacArrayInfo</code>	Query precisions, accumulator widths, PE grid
<code>estimate</code>	M, K, N , dtype	cycles + energy (pJ)	Cost model for compiler-side scheduling

The compiler emits one of `matmul_int8` or `matmul_fp16` for every attention sub-tile, dispatched on the precision controller’s decision. See the worked lowering example in `docs/isa/precision_controller_isa.pdf` §4.1 for the full attention-tile flow.

3. Data Type Semantics

3.1 INT8 Path (Frozen)

- Storage: signed two’s-complement 8-bit (`int8_t`, range $-128 \dots +127$).
- Multiplication: signed 8-bit $\times$ signed 8-bit $\rightarrow$ exact 16-bit product.
- Accumulation: 32-bit signed accumulator (`int32_t`). For K up to 65 536 inputs the accumulator cannot overflow (worst-case $K \cdot 127 \cdot 127 \approx 1.06 \cdot 10^9$, fits in 31 bits).
- No saturation, no requantization at the accumulator boundary; the compiler is responsible for any downcast / requantization step after the matmul.

3.2 FP16 Path (Open — v0.1 Simplification)

- Storage: IEEE-754 binary16 (`half`, 1 sign + 5 exponent + 10 mantissa).
- Multiplication: half $\times$ half $\rightarrow$ round-to-nearest- even $\rightarrow$ half.

- Accumulation: fp32 accumulator (matches typical attention hardware).
- Final output: rounded back to fp16 (round-to-nearest-even).
- **v0.1 caveat:** the reference model uses `float` internally and rounds back to fp16 between operations. The eventual hardware may use a denser accumulator format (e.g., bfloat16) or skip the intermediate rounding. The compiler should not depend on bit-exact parity with v0.1’s FP16 path until v0.2 freezes the choice.

3.3 Why Both Paths (Frozen)

The precision controller decides INT8 or FP16 per tile. The MAC array therefore needs to dispatch to either path on a tile-by-tile basis. A unified mixed-precision path is out of scope for v0.1 because the cost and energy savings of INT8 only materialize when the entire matmul stays in INT8.

4. Shape Support (Frozen)

- Arbitrary $M \times K \times N$ (no power-of-two requirement).
- Recommended tile shape: $64 \times 64 \times 64$ (matches the precision controller’s default tile).
- A typical FlashAttention-style block computes:
 - $Q \cdot K^\top$: $64 \times 64 \times 64$
 - $\text{score} \cdot V$: $64 \times 64 \times 64$
- The reference is shape-agnostic — the compiler picks the shape, the reference produces the correct output.

5. Concurrency and Streaming (Open)

The v0.1 reference is synchronous: `matmul_int8(...)` blocks until the result is in C . The eventual hardware will likely expose:

1. An asynchronous “issue” call that returns a token;
2. A “wait” call on the token;
3. Double-buffered input/output regions so the compiler can pipeline.

For v0.1 the compiler can model these by wrapping the synchronous calls in its own scheduler. We will revisit when the MAC array RTL lands.

6. Cost Model (v0.1 Placeholders)

`MacArrayInfo` exposes:

Table 2: Cost-model parameters baked into v0.1.

Parameter	Value (v0.1)	Notes
<code>pe_grid_m</code>	16	PE grid rows; area-budget estimate
<code>pe_grid_n</code>	16	PE grid cols; area-budget estimate
<code>int8_throughput</code>	256 MACs/cyc	One INT8 MAC per PE
<code>fp16_throughput</code>	64 MACs/cyc	FP16 PE is $\sim 4\times$ INT8 area, so $\frac{1}{4}$ the count
<code>pipeline_depth</code>	16 cycles	Fixed startup latency

`estimate(M, K, N, dtype)` returns:

$$\text{cycles} \approx \left\lceil \frac{M \cdot N \cdot K}{\text{throughput}} \right\rceil + \text{pipeline_depth}$$

$$\text{energy}_{pJ} \approx M \cdot N \cdot K \cdot \text{per_op_energy}_{pJ}$$

Per-op energy values: 0.5 pJ for INT8 MACs and 2.5 pJ for FP16 MACs (rough numbers from published 16FFC accelerator data).

These are **placeholders** for the compiler’s scheduler. The numbers will tighten once the MAC array RTL is written and synthesized; today they are a reasonable order-of-magnitude estimate. The compiler should use them for relative comparison (INT8 vs FP16, which shape is faster) rather than absolute timing.

7. What Is Not in v0.1

- Sparsity acceleration (skip-zero PEs).
- Mixed-precision MAC (e.g., INT4 $\times$ INT8 within a single op).
- Batched matmul as a single op (compiler can loop).
- Hardware fault injection / ECC.
- Power-state management.

These are all out of scope for the reference model until the chip architecture commits to them.

8. Compiler Binding

Same template as the precision controller. Three layers:

1. **C++ class** (`lhsi::mac::MacArray`) for MLIR / TVM C++ backends.
2. **extern "C" API** (`lhsi_mac_*`) for plain-C runtimes and FFI.
3. **Python reference** (`mac_array_ref.py`) for compiler-side prototyping and unit tests.

All three pass identical test vectors. If the C++ and Python disagree, that is a bug — open an issue.

9. Open Questions

Bring these to the next compiler-team sync; the answers shape v0.2.

1. Does your compiler emit individual matmuls, or does it expect a fused `attention(Q, K, V)` op? Affects whether we should add a fused-attention reference operation.
2. Do you need cycle-level cost estimates, or is operation-count estimation sufficient for your scheduler?
3. For FP16: do you expect bit-exact parity with our hardware's FP16 rounding, or just numerically-close output? Affects v0.2's rounding-mode decision.
4. Async issue + token model — do you want this in v0.1 (we would add it now) or v0.3 (after RTL exists)?

10. Verification Status

The v0.1 reference passes the following checks (see `sw/reference_model/test_mac_array_ref.cpp` and the Python equivalent).

Table 3: MAC array v0.1 verification matrix.

Test	Status
INT8 matmul vs. naive int64 reference (random shapes)	exact match
FP16 matmul vs. naive double reference (random shapes)	within $\pm 5 \cdot 10^{-3}$ relative
Edge cases: zero, identity, signed extremes	pass
extern "C" API matches C++ class	pass
fp16 round-trip helpers (float $\leftrightarrow$ bits)	pass
Cost-estimate sanity (FP16 slower / costlier than INT8)	pass

11. Change Log

- `mac-array-v0.1` (2026-05-14): First public draft. Reference model complete in Python, C++ class, and extern "C" shim. Verification matrix in §10 passes. RTL pending.

LonghornSilicon ACU MAC Array Design Specification. This document is open and may be redistributed. The accompanying reference implementations are at `sw/reference_model/mac_array_ref.{hpp,cpp,py}` in `LonghornSilicon/adaptive-precision-attention`.